# Functional Requirements for the ASE Lab 2026/2027 Project

## Monster Tamer Backend

### 1) Introduction

This document defines the project's functional requirements and may be updated if unforeseen situations arise. Most items refer to the lab activities; the course slides are additional sources for the project realisation.

The service was introduced in Lab 2 ("User Stories") and the project guidelines were outlined in Lab 3 ("Project Kick-off"). Some requirements relate to topics that will be explained later in the course. Refer to the "Project take away" slide in each lab for direct links to the relevant lab topics and project details.

The final software must implement all red and yellow user stories for players listed in the "Project backlog" sheet of Monster_Tamers_Backlog.xlsx. Green user stories (or any additional features, including those not listed in the backlog) are optional, but will be rewarded with a higher evaluation if implemented and documented effectively. Achieving the maximum grade (30 cum laude) is possible by implementing only the red and yellow user stories. Following best practices for every tool you use is highly recommended.

Students must complete the backlog by defining verifiable acceptance criteria and identifying dependencies for all red and yellow user stories and for each optional story they implement. Acceptance criteria should describe observable behaviour, preferably using the Given / When / Then format introduced in Lab 2. Dependencies should identify prerequisite user stories by ID, or explicitly state that there are none. Related stories are not necessarily dependent. The completed backlog must be included with the project documentation.

### 2) Architecture & Repository Requirements

The backend must follow a microservices architecture and be deployable with Docker Compose. Running the following must build and start the entire backend architecture:

```
docker compose up
```

Each microservice must expose a REST API, communicate over HTTPS, and persist its state in a data store.
The code must be hosted on GitHub. The repository must include a README.md with a standard Get Started section containing commands to install, build, run the backend, and run the main tests.
When declaring external dependencies, specify the exact version. For example, in Python, each requirements.txt must pin versions (e.g., Flask==3.0.3).

A minimal frontend with a basic user interface must be implemented to allow users to interact with the backend and perform its core functionalities. The frontend must run outside the Docker Compose environment and must not be connected to the Docker network used by the backend microservices. All interactions with the backend must occur exclusively through the HTTPS REST API exposed by the API Gateway in localhost. Direct communication with internal microservices or databases is not allowed, and no relevant business logic should reside in the frontend. The backend must remain fully functional and independently deployable through docker compose up, without requiring the frontend to be running. The README.md must include instructions for launching the frontend separately and connecting it to the API Gateway.

## 3) Documentation (in the docs/ folder of the repo)

Include at least the following:

- Final report in PDF format (based on the template that will be provided by the course).
- OpenAPI specification files for the REST API exposed by the backend and the internal microservices.
- Bruno/Postman compatible collections (JSON files) with tests for at least three internal (no API gateway or DB manager) microservices. Use a subfolder if there are many files.
- Locust file(s) for performance tests.
- A copy of the YAML file(s) specifying GitHub Actions workflow(s).
- The completed project backlog, including students' acceptance criteria and dependencies for the mandatory and any implemented optional user stories.

The microservice architecture must be free of architectural smells, with the exception of the "Endpoint-based Interaction" smell, as discussed in the class on "Architectural smells and Refactoring for Microservices".

## 4) Core Functionalities

Design the backend with client usage in mind. The main business logic must reside in the backend, although a client may combine information retrieved by separate requests (for example, the image and details of a monster). Any assumptions about client-side logic must be documented in the final report.

The main core functionalities are:

- Registration, login and saved progress: each player can register, log in, and return to their own persistent game progress. (See Security for details.)
- Monster catalogue and personal collection: players can inspect all available monsters and those they own. Every monster must have an image and at least two gameplay characteristics that influence encounters. A new player receives an initial collection; additional monsters may be obtained during the game.
- Team preparation: players can organise a team using only monsters in their personal collection. Teams must have a limited size and satisfy the game rules chosen and documented by the group.
- Encounters: players can select a valid team to face an encounter with a wild monster. Different encounters may have different characteristics; the result must depend on the selected team and monster characteristics, and both success and failure must be possible. An encounter does not have to be a turn-by-turn battle. The group must define and document its encounter rules, lifecycle, outcomes, termination conditions and handling of contingencies.
- Results, rewards and progress: completing an encounter produces a result and may award a new monster. The result and any changes to the player's collection or progress must be handled consistently by the backend and saved permanently.
- Encounter history: each player can review the results of previous encounters and inspect their details.

The minimal gameplay must support the core loop: prepare -> encounter -> result -> progress. The chosen monster catalogue, team constraints, encounters, rewards and other gameplay rules are up to each group, provided they satisfy Lab 3 and the mandatory user stories in the backlog. Implementing more complex gameplay rules allows groups to adapt the core loop and core functionalities accordingly.

Important: Communication between microservices must not rely on the client. For example, encounter history and any rewards must be updated by the backend services managing encounter results, not by a client request that posts an arbitrary outcome.

In general, consider and mitigate potential fraudulent behaviours by players and avoid actions that could lead to inconsistent state or game balance.

Logging every event is not mandatory. However, for debugging, you are encouraged to implement appropriate logging.

## 5) Testing

Component tests are required for at least three internal (not the API gateway or any DB manager) microservices. For each endpoint, provide at least:

- one test for valid input (expecting a response code 2xx), and
- one test for invalid input (expecting an error response).

Component tests and Integration tests must be conducted using Bruno/Postman (or a compatible tool), and the collection should be exported as a JSON file. The same Component tests requests can be reused as integration tests against the gateway with the full architecture running (probably changing the base URL).

Performance testing must be conducted using Locust, targeting only the gateway. The performance test should cover the core functionalities.

Tests should also verify the acceptance criteria defined by students in the backlog, including game-rule validation, ownership checks, encounter outcomes and persistence of progress.

## 6) Security

- All containers must run in user mode to prevent privilege escalation on the host.
- All communication must use HTTPS (or TLS), with each microservice utilising self-signed certificates.
- Inputs received by each microservice must be sanitised, and sensitive data must be encrypted in the database(s).
- Authentication and authorisation must comply with OAuth2, using JWT tokens. Authentication uses username and password credentials, which must be transmitted, stored, and validated securely. Follow the specifications indicated in the Access Security lab.
- The codebase must undergo automatic static and dependency analysis using free tools available for the language(s) in use. Docker images must be free from critical and high-severity vulnerabilities. Follow the specifications indicated in the Security Analysis lab.

## 7) Evaluation

Each group member is responsible for the entire project: they must understand the backend's design, deployment, execution, testing, and the rationale behind the realisation. This does not require knowledge of every low-level implementation detail.

Evaluation criteria:

- Correctness & operability: we must be able to install and execute the architecture and perform the core functionalities by following the documentation.
- Documentation quality: compliance with specifications, clarity, usefulness, and appropriate length (not too much, not too little), including the completed backlog.
- Design choices: there is a high degree of freedom; explain your decisions to show they are reasonable and well-justified.
- Testing: tests should sufficiently cover the main expected behaviours of the backend.
- Security: all specified security requirements must be implemented. We will attempt attacks to verify the implementation.
- Additional features (optional): reasonable, well-implemented, and well-documented features can earn up to 3 extra points.

## 8) Miscellaneous

In general, any solution that complies with these functional requirements is acceptable. The "Project backlog" sheet of Monster_Tamers_Backlog.xlsx defines the required and optional user stories; the "Realistic backlog" sheet is not used to determine delivery obligations. If you are unsure about any aspect of your solution, send an email to alessandro.bocci@unipi.it to raise your concern or to arrange an office-hour meeting.